

AwaazIQ

From Citizen Voice to Intelligent Action

Feature Build Plan and Team Work Split

Every feature we have designed or built, the real government data behind each one (checked live, not assumed), exactly how to build it, and how the work divides across four people. Written so each member can pick up a workstream and build it without re-doing the research.

441

villages scored (Kolhapur
306 · Nashik 135)

814

specific assets (schools,
hospitals, road spots, water
points)

9,352

schools with live UDISE+
data available

79 · 65 · 46

tests passing (scoring ·
backend · pipeline)

Snapshot: 14 September 2026 · Branch **submission** · Pilot districts Kolhapur and Nashik, Maharashtra. Every data source marked "verified" returned real data to a live request on this date. Sources marked otherwise say why.

Contents

1. How to use this document
2. What AwaazIQ is, and our white space
3. What is already built and verified
4. Team work split (4 workstreams)
5. Ground rules for everyone
6. Data source catalogue
7. How the priority score works today
8. Workstream A: Data pipelines
9. Workstream B: Scoring engine
10. Workstream C: Photo checks and fake-complaint defence
11. Workstream D: Map, roads and the white-space view
12. Blocked and later features
13. Build order and dependencies
14. Decisions still open
15. Appendix: exact endpoints and code
16. Appendix: sources ruled out, and why

1. How to use this document

- **Section 4** gives each person one workstream. Read your workstream section (8, 9, 10 or 11) in full; it has the data, the files to change, the formulas and the tests.
- **Section 6** is the single list of every data source we checked, with its real status. If a source is not marked **VERIFIED**, do not build on it without re-checking it yourself.
- **Section 15** has the exact URLs, request formats and code snippets, including the encryption trick the water-quality site uses, so you don't have to rediscover them.
- Longer research write-ups live in the repo root: `FEATURE_ROADMAP.md` (master list), `LIVE_GOV_DATA_RESEARCH.md`, `STALE_INFRA_DEFICIT_RESEARCH.md`, `ASSET_LEVEL_PRIORITIZATION_RESEARCH.md`, `PHOTO_VERIFICATION_RESEARCH.md`.

Status labels used throughout: **VERIFIED** real data pulled live · **LIKELY** reachable, data not yet pulled · **BLOCKED** CAPTCHA, login or dead server · **TO BUILD** designed, not started · **WAITING** needs an outside approval.

2. What AwaazIQ is, and our white space

AwaazIQ takes citizen complaints about rural infrastructure (roads, water, health, schools) in Hindi, Marathi, English or a mix, by text or voice. It removes personal details, places each report on the map, groups related reports, and ranks what needs fixing first using a 9-part priority score built mostly from government records.

The white space (from our research report)

Capability E: prioritising citizen demand against government investment data. Our research report (section 9) found this absent in every system checked, in India and globally. Other systems have multilingual intake, maps, or budget data, but none combines citizen demand with investment data to decide where money should go. The report's matching engine should compare demand and investment on **location plus asset type**, with three results: *funded but not delivered*, *demanded but not funded*, and *funded where nobody asked*.

The village → specific-asset view we built is the report's **feature #21, "Grievance-to-Infrastructure Reclassification"** ("which road segment, which health sub-centre catchment"). The report calls it "the deepest structural move in the whole design." It is the unit that Capability E should match against: once each complaint is tied to a specific school, hospital or road, we can ask "was money already sanctioned for *this* asset?"

Where the white space stands today

White-space piece	Status	What's missing
Demand vs investment matching (<code>intelligence/investment.py</code> , "Demand vs Investment" button)	BUILT, BUGGY	Counts every complaint within 5 km (health, school, water, road) against road-only PMGSY works. Result: 762 villages wrongly flagged "demanded but unfunded" (Hajgoli Bk: 141 reports of all kinds vs 0 road works). Fix in B4.
"Already funded here" warning	BUILT	Roads only, and village-level. Only 184 of 552 PMGSY works are linked to a village.
Scheme eligibility (PMGSY / IPHS / RTE / JJM rules)	BUILT	Rule-based, worth +4 points.
What-if budget simulation	PARTIAL	Fixed points per crore, not "people reached per rupee" (B6).
Investment data beyond roads	TO BUILD	Schools: UDISE+ grants are verified (A1). Water: JJM scheme money not yet verified. Health: no per-facility budget exists online. Whole-village budget (GPDP) is CAPTCHA-blocked.

3. What is already built and verified

Feature	Status	Details
Exact report location (map pin, "use my location")	DONE	Commits <code>4b83dff</code> → <code>0040806</code> . Pin must be within 5 km of the chosen village (<code>PIN_MAX_DISTANCE_KM</code>). Pin is never exposed on public endpoints.
Groundwater evidence (NWDP)	DONE	Commit <code>03c26d9</code> . 9 real telemetry stations; shows "groundwater level: X m below ground, trend: falling" on water clusters. Values checked against the live NWDP CSV. Evidence only, no score effect.
Rainfall (MOSDAC)	WAITING	Commit <code>d6ee662</code> . Real dataset, but bulk download needs an ISRO login. Loader exits honestly with no data. An earlier version made up rainfall numbers and was reverted.
Village → specific-asset view	DONE	Commits <code>910142e</code> → <code>4dbe99e</code> , plus same-day fixes (see below). District map shows one bubble per village. One click zooms in and opens the village priority panel. Each school, hospital, road spot and water point has its own pin and its own 9-part score. Village score = its most-in-need asset's score.
Citizen picks the specific school/hospital	DONE	Dropdown in the report form (<code>/gazetteer/facilities</code>).
Demo data	DONE	1,000 sample complaints attached to real registered schools and hospitals, with road/water reports placed around their village (<code>backend/assign_synthetic_assets.py</code> , has <code>--undo</code>). The DEMO labels are hidden in the UI by owner decision; the data still carries <code>is_demo</code> .

Before anyone branches: the village-view fixes made on 14 September are **not committed yet**. They fix: giant white circles, report dots cluttering village mode, zoom-to-fit, pin sizes going wrong after the panel opens, overlapping pins, the "demo-assigned" label, and hidden DEMO badges. They touch `frontend/css/app.css`, `frontend/js/app.js`, `frontend/index.html`, `frontend/report.html`, `intelligence/recompute.py` and `intelligence/test_intelligence.py`. Commit them first, so everyone starts from the same code.

Current database snapshot

Item	Count	Notes
Citizen reports	1,209	1,000 are sample (demo) reports; the rest are real or test submissions.
Category clusters	116	Top score 74.73. Used by what-if and compare.
Specific assets	814	Schools 220, health 177, road 199, water 218.
Villages with a priority	441	Kolhapur 306, Nashik 135.
Named facilities (register)	10,464	9,352 UDISE schools + 1,112 NIC health facilities.
PMGSY road works	552	184 linked to a village by name; 368 unlinked.

4. Team work split (4 workstreams)

The work splits into four workstreams that touch mostly different files, so four people can build in parallel. Each person works on **their own git branch** and merges through a pull request. Never run two people or two AI agents in the same folder at the same time: that caused three rounds of stray, corrupted files on 14 September.

WORKSTREAM A · MEMBER 1

Data pipelines

- A1 UDISE+ school loader: teachers, classroom condition, grants (verified)
- A2 JJM water-testing loader, using the AES method (verified)
- A3 CWC river-level loader (verified)
- A4 SACHET disaster-alert loader (verified)
- A5 Retry MGNREGA, JJM scheme money, Mission Antyodaya

Files: new `backend/load_*.py`, `backend/models.py`, `intelligence/realdata.py` (lookup functions only).

WORKSTREAM B · MEMBER 2

Scoring engine

- B1 Stale-record discount (trust-blend, 10-year half-life)
- B2 Emergency urgency (bridge/building damage, cap 15)
- B3 Corroboration softening for 1–2 reporter assets
- B4 Fix the demand-vs-investment category bug
- B5 Use UDISE+ data in the school score
- B6 What-if: people reached per rupee

Files: `intelligence/scoring.py`, `config.py`, `recompute.py`, `investment.py`, `whatif.py`, tests.

WORKSTREAM C · MEMBER 3

Photo checks and fake-complaint defence

- C1 Camera-only capture with GPS and time at capture
- C2 Count distinct accounts, not wording
- C3 Duplicate-photo check (`imagehash`)
- C4 Edit detection (error-level analysis)
- C5 Pothole model (team's own) as road evidence
- C6 Building/bridge damage model for emergencies
- C7 Screen-replay (moiré) check

Files: `frontend/js/report.js`, `backend/routes_citizen_report.py`, new `backend/photo_checks.py`, model folder.

WORKSTREAM D · MEMBER 4 (OWNER)

Map, roads and the white-space view

- D1 Task 5: PMGSY GeoSadak real road names
- D2 Show funding status on village and asset panels
- D3 Map polish: label overlap, panel height, state view
- D4 Demo script, sample-data disclosure, final test pass

Files: `frontend/js/app.js`, `panels.js`, `css/app.css`, new `backend/load_pmgsy_geosadak.py`.

Shared contracts, so the pieces fit. New data shows up in two places only: (1) a `*_lookup` function in `intelligence/realdata.py` that returns `(value, evidence_dict)` or `(None, {})` when there is no data; (2) a named key in the asset/cluster evidence JSON (for example `evidence["school_condition"]`, `evidence["water_testing"]`, `evidence["hazard_alert"]`). Workstream A agrees the key names with B and D before starting. Every tunable number goes in `intelligence/config.py` with a comment explaining it.

5. Ground rules for everyone

1. **Never make up data.** If a source can't be fetched, return `None` and mark the feature "found, not confirmed". This already happened once: a rainfall loader invented numbers with a formula and labelled them as satellite data. It was reverted. Demo complaints are the only sample data allowed, and they must stay flagged `is_demo`.
2. **Check it live before building on it.** A search result is not proof. Mission Antyodaya looked real in search results, but its domain is dead. Pull at least one real row for Kolhapur or Nashik before writing a loader.
3. **Never bypass a CAPTCHA or a login.** eMARG and eGramSwaraj are off-limits for scraping for this reason.
4. **Scrape politely:** at most about 1 request per second, an honest User-Agent, cache results, and refresh on a schedule (UDISE yearly, JJM monthly, CWC and SACHET every 15–60 minutes).
5. **No complaint is ever auto-rejected.** Verification signals only raise or lower confidence and flag reports for an officer. A score may be damped, never zeroed.
6. **The score must stay explainable:** the 9 breakdown numbers must add up to the priority score. A test enforces it; don't break it.
7. **Tests before and after every change:** `python -m intelligence.test_intelligence` (79/79), `cd backend && python test_citizen_report_endpoint.py` (65/65; it writes to the database, so back up `backend/hackathon.db` first), `cd pipeline && python test_pipeline.py` (46/46).
8. **After every commit run `git status`.** Only untracked research files may remain. If a tracked file shows as modified after you committed, an editor wrote an old copy back; fix it before moving on.
9. **When you change any frontend file, bump the version tag** in `index.html` / `report.html` (currently `?v=village5`). Otherwise browsers keep the old file and you'll think your change didn't work.
10. **Privacy stays as built:** citizen identity lives only in `citizen_identity`, exact pins are never returned by public endpoints, and pin-based asset coordinates are rounded to 3 decimals (about 110 m).

6. Data source catalogue

Every source we looked at, with what it gives and whether we can use it. Exact request formats are in section 15.

Verified: real data pulled live

Source	What it gives	Access	Used for
UDISE+ Know Your School kys.udiseplus.gov.in/web-app/api/	Per school, 2024-25 : teachers (regular/contract/part-time), classrooms by condition (good / minor repair / major repair), toilets, drinking water, electricity, boundary wall, grant received and spent .	JSON API, no login, no CAPTCHA, by 11-digit UDISE code (we hold all 9,352). School search is CAPTCHA'd but not needed.	Education infra_deficit per school asset (A1, B5); school investment (D2). Test: Vidya Mandir, Mhalunge Khalsa: 5 teachers, 4 good + 1 minor-repair classroom, ₹25,000 grant, ₹25,000 spent.
JJM WQMIS water testing ejalshakti.gov.in/WQMIS	Per village, current year 2026-27 : number of water samples tested and for which parameters (pH, FRC, turbidity, TDS, hardness, arsenic/fluoride/nitrate/iron), and villages where testing wasn't done.	ASP.NET forms. Every value is AES-encrypted with a fixed key found in the site's own JavaScript. Replicated in Python.	Water infra_deficit and evidence (A2). Test: Ardal village, Kolhapur: 30 samples tested this year. Kolhapur has 12 blocks, Nashik 15.
CWC flood forecasting ffs.india-water.gov.in/iam/api/	168,019 stations; live river level and rainfall readings.	Open JSON API. Needs a <code>class-name</code> header and a JSON filter.	Emergency corroboration (A3, B2). Test: Nashik station <code>MHSWIWRM026</code> read at 17:00 on 14 Sept. The station named "KOLHAPUR" is dead (last reading 2017, wrong coordinates).
NDMA SACHET alerts sachet.ndma.gov.in	Live disaster alerts (heavy rain, flood, lightning) from IMD, CWC and the state, by district.	Public RSS feed <code>rss_maharashtra.xml</code> , each item links to a CAP XML message.	Emergency corroboration (A4, B2). Test: 10 current Maharashtra alerts in Marathi. Alerts expire, so store history.
NWDP groundwater nwdp.nwic.gov.in	Groundwater level telemetry, Maharashtra GW department.	Direct CSV download, no login.	ALREADY LOADED 9 stations, water evidence.
PMGSY works pmgsy.dord.gov.in	552 road works: sanctioned cost, status, year.	Dashboard API with session token.	ALREADY LOADED Road deficit, "already funded".
PMGSY GeoSadak github.com/datameet/pmgsy-geosadak	Road geometry shapefiles (<code>Road_DRRP</code>), road type and surface.	GitHub mirror (Government Open Data Licence). The official portal's domain is dead.	Real road names for road pins (D1).
Census 2011, BharatNet 2022, LGD, NIC HealthGIS	Village amenities, fibre status, admin hierarchy, health facility locations.	Bulk files.	ALREADY LOADED The current score's base.

Source	What it gives	Access	Used for
NFHS-5 (2019-21)	District health and nutrition indicators.	DHS Maharashtra PDF downloads directly.	Fresher health context (district-level only).

Likely, blocked, or waiting

Source	Status	Why, and what to do
JJM contamination results (Contaminantwise, lab reports)	LIKELY	Same site and same AES method as the water-testing report; not yet pulled. Would give actual unsafe-water findings.
JJM scheme money (cost, spending, status)	LIKELY	Same ejalshakti site. Would give water investment data for the white space. Not verified.
MGNREGA works per panchayat	SERVER DOWN	State page works (Maharashtra code 18), but every report server returned 503 or refused the connection. Retry later.
eGramSwaraj GPDP (whole-village budget plan)	CAPTCHA	The richest village budget source (every planned work, cost, sector, Finance Commission grants), but every report page has a CAPTCHA. Download a few showcase villages by hand, or request data from the Panchayati Raj department.
eMARG road inspections	CAPTCHA	PMGSY's own current road-condition inspections. Public lookup is CAPTCHA'd. Request data from NRIDA.
MOSDAC GSMaP rainfall	LOGIN	Needs an ISRO MOSDAC account: ordinary email sign-up, approval sent by email. Then the loader can be finished.
Planet Labs imagery (3.7 m)	APPLICATION	Free Education & Research programme (university email, 3,000 km ² /month). For bridge/road-collapse confirmation only.
Mission Antyodaya (yearly village survey)	UNREACHABLE	Old domain dead; new domain resolves but won't connect from here. Retry from another network.
Health budgets (NHM)	NOT ONLINE	Only state/district PDFs. Fallback: NHM's fixed yearly allowance per facility, clearly labelled as an entitlement, not spending.

7. How the priority score works today

Every cluster and every asset gets the same 9-part score (0–100). Four "need" terms are each 0–1 and weighted 25%:

```
gap points = 81 × confidence_gate × (0.25·demand + 0.25·population + 0.25·infra_deficit +  
0.25·vulnerability)  
priority   = gap points + equity (0–10) + strategic (0–4) + urgency (0–3) + feasibility (0–2) –  
cost penalty (0–3)
```

Term	What feeds it now	Planned change
demand	Distinct reporters (by wording), capped at 25	Count distinct accounts (C2); soften below 3 reporters (B3)
population	Census population in the category catchment (road 5 km, water 3, health 8, school 4), log-scaled	—
infra_deficit	Government records (Census 2011, PMGSY, JJM tap coverage, IPHS norms); citizen severity only if no record exists	Stale-record discount (B1); UDISE+ per-school data (B5); JJM testing (A2)
vulnerability	Census: distance to HQ, power hours, drainage, internet	Stale-record discount (B1)
equity	BharatNet fibre down = hard to report from here (+10)	—
strategic	Qualifies under a published scheme rule (+4)	—
urgency	Flat +3 for every road and water complaint	Replaced by emergency score, cap 15 (B2)
feasibility	Near a block HQ (+2)	—
cost penalty	Larger catchments cost more (–3)	—

Why a village can rank #1 on one complaint. Halewadi (Kolhapur) scores 68.18 with only 1.62 points from complaints. The rest comes from the government record: 2 doctor posts sanctioned, 0 filled; 46,992 people within 8 km; 84 km from the district HQ; fibre down in all 16 nearby villages. That is the "need, not noise" design working as intended. But the old 3-report minimum was dropped in the asset view, so one complaint (possibly fake) can reach #1. B3 fixes that. And health tends to lead the ranking because its catchment (8 km) is the largest.

8. Workstream A: Data pipelines

Goal: bring in the four verified live sources as clean, cached, honest data that the scoring engine and the map can use. Every loader follows the pattern of `backend/load_nwdp_groundwater.py`: fetch, parse, store in a table, save a snapshot file, and print a clear summary. On failure it stores nothing and exits with code 1.

A1 • UDISE+ school loader

Size: M • Priority: 1 • **VERIFIED**

Why	Current (2024-25) condition, staffing and money for each specific school, replacing the 2011 fact "this village has a middle school". The biggest data upgrade we found.
Data	<code>GET /web-app/api/school/report-card?udiseSchCode=<code> and /school/facility?udiseSchCode=<code></code> . Codes come from <code>public_facility.external_id</code> where <code>source='udise'</code> .
New table	<code>school_condition</code> : <code>facility_id</code> , <code>udise_code</code> , <code>year_desc</code> , <code>teachers_regular</code> , <code>teachers_contract</code> , <code>teachers_part_time</code> , <code>classrooms_total</code> , <code>classrooms_good</code> , <code>classrooms_minor_repair</code> , <code>classrooms_major_repair</code> , <code>toilet_boys_functional</code> , <code>toilet_girls_functional</code> , <code>drinking_water</code> , <code>electricity</code> , <code>boundary_wall</code> , <code>total_grant</code> , <code>total_expenditure</code> , <code>fetches_at</code>
Fields	report-card: <code>yearDesc</code> , <code>tchReg</code> , <code>tchCont</code> , <code>tchPart</code> , <code>totalGrant</code> , <code>totalExpenditure</code> (their spelling). facility: <code>clsrmsInst</code> , <code>clsrmsGd</code> , <code>clsrmsMin</code> , <code>clsrmsMaj</code> , <code>toiletbFun</code> , <code>toiletgFun</code> , <code>drinkWaterYn</code> , <code>electricityYn</code> , <code>bdrywallType</code> .
Volume	9,352 schools × 2 calls ≈ 18,700 requests. At 1 request per second that's about 5 hours: run it once, resumable (skip codes already fetched), refresh yearly.
Lookup	<code>realdata.school_condition(facility_id) → (value, evidence)</code> .
Tests	Parse a saved JSON sample into the table row; missing fields become None, not 0; resume skips existing codes.

A2 · JJM water-testing loader

Size: M · Priority: 2 · VERIFIED

Why	Current water-testing coverage per village, replacing 2011 Census tap columns. Villages where water was never tested are a real deficit signal.
Method	1) GET <code>/WQMIS/Report/rpt_ftk_testing_village</code> to get a session cookie and <code>__RequestVerificationToken</code> . 2) Encrypt every value with AES-128-CBC, key = IV = <code>8080808080808080</code> , PKCS7, base64 (snippet in 15.2). 3) POST <code>/WQMIS/Common/District_Bind_without_session</code> → blocks → panchayats → villages. 4) POST <code>/WQMIS/Report/Get_rpt_ftk_testing_village</code> .
Key trick	Results come back one level below the filter you send. To get per-village rows, send district + block + panchayat with <code>villid=0</code> . Sending a specific village returns nothing.
IDs	Maharashtra 18 · Kolhapur 285 · Nashik 277 · e.g. Ajara block 3264, Ardal panchayat 101418, Ardal village 240666. Financial year format <code>2026-2027</code> .
New table	<code>water_testing</code> : <code>fin_year</code> , <code>district_id</code> , <code>block_id</code> , <code>gp_id</code> , <code>village_id</code> , <code>village_name</code> , <code>samples_tested</code> , <code>ph</code> , <code>frc</code> , <code>turbidity</code> , <code>tds</code> , <code>hardness</code> , <code>ar_fl_ni_iron</code> , <code>villages_not_tested</code> , <code>gazetteer_id</code> (match village names to our gazetteer like the existing JJM loader).
Next	Same method on <code>/WQMIS/Report/Contaminantwise</code> to get actual unsafe-water findings. Check it live first.
Tests	AES output for "18" equals <code>4rB5wpBtmBBmFEyZRZokg==</code> ; parser handles the "--Select--" placeholder rows.

A3 · CWC river-level loader

Size: M · Priority: 4 · VERIFIED

Why	A river in flood near a "bridge collapsed" report is strong independent confirmation of an emergency.
Data	GET <code>/iam/api/layer-station/specification/sorted-page</code> (header <code>class-name: LayerStationDto</code>) to find stations; <code>/layer-station-geo/...</code> (<code>LayerStationGeoDto</code>) for coordinates; <code>/new-entry-data/specification/sorted-page</code> (<code>NewEntryDataDto</code>) for readings, sorted by <code>id.dateTime</code> DESC.
Step 1	Build a station list for Kolhapur (Panchganga/Krishna basin) and Nashik (Godavari). Keep only stations with a reading in the last 7 days and coordinates inside the district. The "KOLHAPUR" station fails both checks.
Step 2	Map the reading codes (<code>GBD</code> , <code>GBT</code> , <code>GEP</code> , ...) to "water level", "rainfall" etc. from the site's own station pages. Don't guess.
Table	<code>river_reading</code> : <code>station_code</code> , <code>name</code> , <code>lat</code> , <code>lon</code> , <code>datatype_code</code> , <code>value</code> , <code>observed_at</code> . Poll hourly.
Lookup	<code>realdata.hazard_near(lat, lon, hours=72)</code> → (<code>flag</code> , <code>evidence</code>), shared with A4.

A4 · SACHET disaster-alert loader

Size: S · Priority: 3 · VERIFIED

Data	<code>https://sachet.ndma.gov.in/cap_public_website/rss/rss_maharashtra.xml</code> . Each <code><item></code> links to <code>FetchXMLFile?identifier=...</code> , a CAP message with event, severity, urgency, area/district and expiry.
Build	Poll every 15 minutes, fetch each new CAP message, store <code>hazard_alert: identifier, event, severity, urgency, area_desc, districts, effective, expires, sender</code> . Keep history, because alerts expire and we need "was there an alert 2 days ago?"
Match	By district name in the area description (text is often Marathi; keep a small name map for Kolhapur/कोल्हापूर, Nashik/नाशिक).

A5 · Retry list

Size: S each · Priority: 5

- **JJM scheme money** (cost, spending, status per village) on ejalshakti: would give the white space water investment data.
- **MGNREGA** panchayat works: retry the report servers on a different day or network.
- **Mission Antyodaya**: retry `missionantyodaya.dord.gov.in` from another network; if reachable, inspect columns before building.
- **MOSDAC**: once someone registers an ISRO account, finish `backend/load_mosdac_rainfall.py` with a login session.

9. Workstream B: Scoring engine

Goal: make the score react to current reality, not only 2011 records, while staying explainable and hard to fake. All changes go in `intelligence/`, constants in `config.py`, tests in `test_intelligence.py`.

B1 • Stale-record discount (trust-blend)

Size: M · Priority: 1 · decided

Problem it fixes: in the Shinganapur test, 22 people reported a collapsed bridge but the cluster ranked #63 of 114, because Census 2011 says the road is fine and a government record always wins. Now an old record loses weight only when a real burst of recent, severe, independent reports contradicts it.

```
burst_ratio    = (distinct reports in last 72 h / 72) ÷ (all distinct reports / hours since
first report)
velocity       = 0 if burst_ratio ≤ 1, else min(1, ln(1 + burst_ratio) / ln(1 + 20))
freshness      = 0.5 ^ (record_age_years / 10)          # half-life 10 years (owner decision)
contradiction  = velocity × share_of_reports_with_high_severity
trust          = 1 - (1 - freshness) × contradiction
infra_deficit  = trust × government_value + (1 - trust) × citizen_severity_value
```

- Add `created_at` to `_load_reports()` in `recompute.py`; count **distinct reporters**, not rows (anti-spam).
- Constants: `BURST_WINDOW_HOURS = 72` (already exists), `VELOCITY_RATIO_CEILING = 20`, `RECORD_TRUST_HALF_LIFE_YEARS = 10`.
- Record age: Census = current year – 2011 (compute it, don't hardcode 15); PMGSY = sanction year; UDISE = 2024; JJM testing = current year.
- Apply to `real_infra_deficit` and `real_vulnerability` in all four categories. The burst signal does **not** go into urgency.
- Show a third evidence state in the panels: "blended: 48% government record (2011, 15 yrs) + 52% citizen reports".
- **Worked example:** Census record 15 years old → freshness 0.354. Shinganapur contradiction ≈ 0.8 → trust ≈ 0.48 → about 10 of the 20 lost `infra_deficit` points come back.
- **Tests:** no burst + old record → record wins; real burst + old record → blend shifts; real burst + fresh record → record still wins; identical copy-pasted reports don't create a burst.

B2 • Emergency urgency

Size: L • Priority: 2 • decided

Owner's definition: urgency means an emergency only, meaning bridge or building damage **including cracks**, graded crack < partial damage < collapse. It is not for potholes, missing staff or water supply (those use demand, infra_deficit and B1). There is no separate emergency list; it lives inside the score.

```
urgency = URGENCY_MAX × grade × confidence          # replaces today's flat +3
grade    : crack 1/3 • partial damage 2/3 • collapse 1    (from the damage model or
catastrophic wording)
confidence : 0-1, built from independent signals (none decisive alone):
              damage-model score on the photo (C6) • catastrophic wording • burst of reports
(B1)
              • active SACHET alert for the district (A4) • river in flood nearby (A3) •
satellite later
URGENCY_MAX = 15, GAP_SCORE_MAX_POINTS = 69    (was 3 and 81; total stays 100)
```

Why 15: tested against the real 116 clusters. A mid-ranked cluster with a confirmed collapse jumps to **#8** (partial damage #23), while 9 of the current top 10 stay in the top 10 and clusters move about 4 places on average. A cap of 9 only reaches #28 for a collapse; 20 lets emergencies dominate (#4).

- Update `test_breakdown_sums_to_score` and `test_score_stays_within_scale` for the new caps.
- Evidence key `evidence["emergency"]` lists each signal and its contribution, so officers see why.
- Build the term with every signal optional (missing = 0). It works on wording and bursts first, and gets stronger as A3, A4 and C6 land. No fake inputs.

B3 • Corroboration softening

Size: S • Priority: 3 • needs owner decision

The old clusters needed 3 independent reports to be ranked; the asset view needs 1, so a single complaint can reach #1. **Recommended option:** an asset with fewer than 3 distinct reporters keeps a reduced share of its need points (1 reporter 60%, 2 reporters 80%, 3+ 100%) and shows an "unconfirmed" badge. Implement as an extra factor on the confidence gate in `score_cluster` so the breakdown still sums. Alternative: fewer than 3 reporters means the asset is not ranked and goes to a separate "unconfirmed" list.

B4 • Fix the demand-vs-investment category bug

Size: S • Priority: 1

- In `intelligence/investment.py` `build_village_investment()`, count only reports whose `issue_category` matches the works' category (PMGSY = `road`). Today it counts all categories.
- Return the category in each row, so when school (A1 grants) and water (A5) money arrives the same code handles them.
- Re-check the headline counts (currently 98 funded-undelivered, 762 demanded-unfunded, 37 funded-not-demanded). They should drop sharply. Remember the demand side is sample data: mismatches are a demonstration, not findings.

B5 · Use UDISE+ in the school score

Size: M · Priority: 2 · needs A1

```
school_deficit = max( major_repair_classrooms / classrooms_total,
                      teacher_shortfall = max(0, 1 - teachers / RTE_required_teachers),
                      no_drinking_water, no_functional_toilet )      # all 0-1
```

- For education assets tied to a UDISE school, use this as the asset's `real_infra_deficit` instead of the village share. Record age = 2024 (fresh).
- RTE teacher norm: use enrolment from the report card if present; otherwise report the teacher count as evidence only. Don't invent enrolment.
- Evidence key `evidence["school_condition"]`: teachers, classroom condition, grant received/spent, year.

B6 · What-if: people reached per rupee

Size: M · Priority: 4

Replace "0.4 points per crore" (`whatif.py`) with the research report's version: given a budget, pick assets in priority order until the money runs out, using real unit costs (the PMGSY median cost per km already computed in `realdata.cost_benchmark`; school grants from A1), and report the **population reached per rupee**. Show which assets get funded and which just miss out.

10. Workstream C: Photo checks and fake-complaint defence

Three separate questions, three layers. None of them rejects a complaint; each raises or lowers confidence and flags it.

Layer	Question it answers	How
1 · Corroboration	Did several independent people report it recently?	Already built (cap 25, 3-report clusters) plus B1, B3, C2
2 · Photo is genuine	Is it a real, new photo taken here, not edited, reused or shown on a screen?	C1, C3, C4, C7
3 · Photo shows the problem	Does the photo actually show a pothole, crack or collapse?	C5, C6

This mirrors how insurance companies check photo claims: metadata, edit detection, reverse image search, and an AI-image check used only to flag for review. Staffing complaints ("no teacher", "no doctor") can never be photo-verified; only corroboration backs them up.

C1 · Camera-only capture, GPS and time at capture

Size: M · Priority: 1

- In `frontend/js/report.js`, capture photos with `navigator.mediaDevices.getUserMedia` + the Image Capture API instead of `<input type="file">`. `capture="camera"` is only a hint and still allows gallery uploads on many phones.
- Take a short burst of 3–5 frames, not one still (used by C7), and read GPS at the same moment (reuse the Task 1 geolocation button).
- Needs HTTPS (or localhost), same as geolocation.
- Backend: store `capture_lat/lon` and `captured_at` per attachment; flag if the capture point is more than 5 km from the chosen village.

C2 · Count distinct accounts

Size: S · Priority: 1

Reports already carry `user_id`. In `recompute.py`, count distinct reporters as distinct accounts (fall back to distinct wording only for anonymous/old reports). One person filing the same complaint in different words then counts once.

C3 · Duplicate-photo check

Size: S · Priority: 2

Use `imagehash` (v4.3.1, installs cleanly with Pillow). Store a perceptual hash per attachment; flag any new photo within a small Hamming distance of an existing one from a different village or asset. This catches the same photo reused, even resized or recompressed.

C4 • Edit detection (error-level analysis)

Size: S • Priority: 3

Free and local, no API. Re-save the JPEG at a known quality, compare with the original, and flag regions with unusually different error levels (a sign of pasting or retouching). Output a 0–1 "possibly edited" score as evidence for the officer.

C5 • Pothole model as road evidence

Size: M • Priority: 2 • team already has the model

- Run the team's pothole model on road photos in a small service (`backend/photo_checks.py`). YOLOv8n-size models run on a normal CPU.
- Output: detections with confidence. Store as `evidence["photo_defect"]` . It feeds road **condition** (as extra citizen-severity weight in B1's blend), **not urgency**.
- Honest limit: the best published road-damage models reach about 77% F1 (RDD2022 benchmark, which includes Indian roads). Treat the output as a signal, never a verdict.

C6 • Building and bridge damage model (for emergencies)

Size: L • Priority: 3

- **Buildings: realistic.** A published ground-photo classifier (ResNet50) sorts not damaged / damaged / collapsed at 93.5%. GitHub starting points: `najmulmowla1/Earthquake-Building-Damage-Detection` (cracked / partially damaged / collapsed). Crack detection (SDNET2018, 56,000 images) reaches 95%+ but only answers "is there a crack", not "is it safe".
- **Bridges: no ready model.** Published bridge models detect cracks and corrosion on close-up inspection photos and are weak (mAP50 0.445). A collapse model needs collected, labelled examples.
- Output the grade (crack / partial / collapse) and confidence for B2.

C7 • Screen-replay check

Size: M • Priority: 4 • research needed

Someone could photograph a picture shown on another screen. Photographing a screen leaves **moiré patterns** (interference between the screen's pixel grid and the camera sensor), screen glare and bezel edges. Research models exist (CNN/Vision Transformer "recapture detection"); KYC companies use the same idea. Cheap first step with C1's burst: a real handheld scene shows small movement between frames; a screen is suspiciously flat. Use it as a signal only.

11. Workstream D: Map, roads and the white-space view

D1 • Real road names (Task 5, PMGSY GeoSadak)

Size: M · Priority: 1

- Download the Maharashtra `Road_DRRP` shapefile from github.com/datameet/pmg-sy-geosadak. **First inspect its attribute table** for Kolhapur and Nashik and report which fields exist (name, surface, category). Don't commit large shapefiles.
- `backend/load_pmg-sy-geosadak.py` → table `pmg-sy_road_segment` (road id/code, name if present, category, surface, a list of points).
- `realdata.load_road_segment_index(db)` and `realdata.nearest_road_segment(lat, lon, segments, max_distance_m=150)`. Then road asset naming prefers the nearest segment's name over "Road near X". Missing fields are None, never invented.
- Also links the 552 PMGSY works to specific roads instead of village names, which makes "already funded" road-specific.

D2 • Show funding status on village and asset panels

Size: M · Priority: 1 · after B4

- On each asset panel, show the matching investment line for that asset's type: roads from PMGSY (and D1), schools from UDISE+ grants (A1), water from JJM scheme money when A5 lands.
- Example: "₹729 lakh sanctioned since 2006 across 4 works, not delivered" on the road asset; "School grant ₹25,000 received, ₹25,000 spent (2024-25)" on the school asset.
- On the village panel, a one-line summary per category: funded-undelivered / demanded-unfunded / no mismatch. This connects the asset view to the white space.

D3 • Map polish

Size: S-M · Priority: 3

- Labels of the top-3 pins can still overlap when pins are very close: add simple label collision (hide a label that overlaps a higher-ranked one).
- The map strip above the detail panel is short (the panel takes about two-thirds of the screen): add a drag handle or a smaller default panel height.
- Optional: village bubbles at state level (today villages appear only after entering a district).

D4 • Demo and disclosure

Size: S · Priority: 2

- DEMO labels are hidden by owner decision, so the pitch must say the pilot runs on sample complaints. The "Halewadi #1" ranking is a demonstration, not a finding.
- Prepare 3 demo walk-throughs: (1) district → village → asset; (2) an "already funded here" road; (3) the Shinganapur-style bridge case before and after B1/B2.
- Final pass: all three test suites green, `git status` clean, version tag bumped.

12. Blocked and later features

Feature	Blocked on	Plan once unblocked
Satellite confirmation of disasters	Planet Labs Education & Research application (university email)	Only for catastrophic road/bridge claims that already have a burst of reports and contradict a clean record. Before/after image comparison near the asset. Never for potholes, staffing or water (a school building is too small even at 3.7 m).
Rainfall evidence	ISRO MOSDAC account	Finish the loader with a login session; show "X mm in the last 72 h" on road and water evidence.
Whole-village budget (GPDP)	CAPTCHA on every report page	Hand-download a few showcase villages; request bulk data from the Panchayati Raj department.
Road condition inspections (eMARG)	CAPTCHA	Formal data request to NRIDA; would give current condition of PMGSY roads.
Reverse image search	Paid API (Google Vision, TinEye)	Catches photos stolen from the web; decide on cost later.

13. Build order and dependencies

Phase	A · Data	B · Scoring	C · Photos	D · Map
0 · Day one	Commit the pending village-view fixes on <code>submission</code> . Everyone branches from that commit. Agree evidence key names (section 4).			
1	A1 UDISE+ loader A4 SACHET loader	B4 category bug B1 stale-record discount	C2 distinct accounts C1 camera capture	D1 GeoSadak roads
2	A2 JJM water testing A3 CWC stations	B5 UDISE+ in score (needs A1) B3 softening (needs decision)	C3 duplicate photos C5 pothole model	D2 funding on panels (needs B4, A1)
3	A5 retries (JJM money, MGNREGA)	B2 emergency urgency (uses A3, A4, C6 when ready) B6 what-if per rupee	C6 damage model C4 edit detection	D3 polish
4	C7 screen-replay research · D4 demo walk-throughs · full test pass · merge to <code>submission</code> .			

Hard dependencies: B5 needs A1 · D2 needs B4 (and A1 for schools) · B2 is built with optional inputs and gets stronger as A3, A4 and C6 land · B1 and B2 both change `scoring.py`, so the same person (Member 2) owns both to avoid merge conflicts.

14. Decisions still open

Decision	Options	Recommendation
Assets with only 1–2 reporters (B3)	(a) keep a reduced share of need points + "unconfirmed" badge; (b) don't rank them	(a): damp, never erase
Health budget fallback	Use NHM fixed per-facility entitlements, or show "no data"	Entitlements, clearly labelled "entitled, not spent"
Should shared hospitals count fully for every village they serve?	Yes (today), or split by population share	Keep for now; revisit if health keeps dominating the top 10
Register ISRO MOSDAC account; apply to Planet Labs	Owner or a team member with a university email	Do both early; approvals take time
Formal data requests (Panchayati Raj for GPDP, NRIDA for eMARG)	Send now, or skip for the hackathon	Send now; mention in the pitch as the path to full coverage

Already decided (don't reopen): village score = most-in-need asset · urgency = bridge/building emergencies only, graded, inside the score, cap 15 · stale-record half-life 10 years · nothing fabricated · no auto-rejection · satellite for disasters only · DEMO labels hidden in the UI, data still flagged.

15. Appendix: exact endpoints and code

15.1 UDISE+ school data

```
import requests
H = {"User-Agent": "AwaazIQ/1.0 (+contact)", "Accept": "application/json",
     "Referer": "https://kys.udiseplus.gov.in/"}
B = "https://kys.udiseplus.gov.in/web-app/api/"
code = "27341212203" # 11-digit UDISE code from public_facility.external_id
card = requests.get(B + f"school/report-card?udiseSchCode={code}", headers=H, timeout=30).json()["data"]
fac = requests.get(B + f"school/facility?udiseSchCode={code}", headers=H, timeout=30).json()["data"]
# card: yearDesc '2024-25', tchReg 5, tchCont 0, tchPart 0, totalGrant 25000.0, totalExpenditure 25000.0
# fac : clsrmsInst 5, clsrmsGd 4, clsrmsMin 1, clsrmsMaj 0, toiletbFun 1, toiletgFun 1, drinkWaterYn 1 ...
# PDF report card: B + f"reportCardPdfByUdiseCode/{code}"
```

15.2 JJM WQMIS water testing (AES method)

```
import re, base64, requests
from Crypto.Cipher import AES # pip install pycryptodome
from Crypto.Util.Padding import pad
KEY = IV = b"8080808080808080" # hardcoded in the site's frmValidate.js
def enc(v):
    return base64.b64encode(AES.new(KEY, AES.MODE_CBC, IV).encrypt(pad(str(v).encode(), 16))).decode()

BASE = "https://ejalshakti.gov.in/WQMIS"
s = requests.Session(); s.headers["User-Agent"] = "AwaazIQ/1.0 (+contact)"
page = s.get(f"{BASE}/Report/rpt_ftk_testing_village", timeout=30)
token = re.search(r'name="__RequestVerificationToken"[^>]*value="([^"]*)"', page.text).group(1)
def common(path, **data):
    return s.post(f"{BASE}/Common/{path}", data={"__RequestVerificationToken": token, **data},
                  headers={"X-Requested-With": "XMLHttpRequest"}, timeout=25).json()

districts = common("District_Bind_without_session", state_id=enc(18)) # Kolhapur 285, Nashik 277
blocks = common("Block_Bind_without_session", state_id=enc(18), District_id=enc(285))
gps = common("GetGramPanchayat_Bind_without_session", state_id=enc(18),
             District_id=enc(285), Block_id=enc(3264)) # rows: JJM_PanchayatId,
PanchayatName
rows = s.post(f"{BASE}/Report/Get_rpt_ftk_testing_village", headers={"X-Requested-With": "XMLHttpRequest"},
             data={"__RequestVerificationToken": token, "draw": 1, "start": 0, "length": 100,
                  "finY": enc("2026-2027"), "stid": enc(18), "dtid": enc(285), "blid": enc(3264),
                  "gpId": enc(101418), "villid": enc(0), # villid=0 -> one row PER VILLAGE of this
                  "type_hgjv": enc(0), "type_ftkLab": enc(0), "type_household": enc(0)})
# Ardal (240666): Nos_of_Sample_tested 30, pH 1, FRC 1, Turbidity 1, TDS 1,
# Nos_of_Villages_FTK_testing_not_carried 0
# Skip placeholder rows whose name is "--Select--".
```

15.3 CWC flood forecasting

```

import json, requests
B = "https://ffs.india-water.gov.in/iam/api"
def spec(field, op, value): # operators: eq neq gt gte lt lte in nin btn like null empty member notmember
    return json.dumps({"where": {"expression": {"valueIsRelationField": False,
        "fieldName": field, "operator": op, "value": value}}})
# stations whose name contains NASHIK ("like" means contains; do NOT add % wildcards)
st = requests.get(f"{B}/layer-station/specification/sorted-page",
    headers={"Accept": "application/json", "class-name": "LayerStationDto"},
    params={"specification": spec("name", "like", "NASHIK"), "sort-criteria": "{}",
        "page-number": 0, "page-size": 20}, timeout=40).json()
# latest readings for one station
sort = json.dumps({"sortOrderDtos": [{"sortDirection": "DESC", "field": "id.dateTime"}]})
rd = requests.get(f"{B}/new-entry-data/specification/sorted-page",
    headers={"Accept": "application/json", "class-name": "NewEntryDataDto"},
    params={"specification": spec("id.stationCode", "eq", "MHSWIWRM026"), "sort-criteria": sort,
        "page-number": 0, "page-size": 5}, timeout=40).json()
# coordinates: /layer-station-geo/specification/sorted-page with class-name LayerStationGeoDto
# count all stations: /layer-station/specification/count?specification={} -> 168019

```

15.4 SACHET alerts

```

Feed : https://sachet.ndma.gov.in/cap_public_website/rss/rss_maharashtra.xml (all-India: rss_india.xml)
Item : <link>https://sachet.ndma.gov.in/cap_public_website/FetchXMLFile?identifier=<id></link>
CAP : read event, severity, urgency, certainty, areaDesc, effective, expires, senderName
Note : expired messages return "No Active Alert" -> store every alert when first seen.

```

15.5 Other verified endpoints

Source	Endpoint
NWDP groundwater CSV	https://nwdp.nwic.gov.in/dataset/85bb464f-bcd5-440b-be2c-fadb2725df4c/resource/817c6429-5853-4198-99e1-20ddf3d9f3bc/download/gwl_tel_6_hourly_maharashtra-gw_mh_1991_2020.csv (2021–2025 stream: https://nwdp.nwic.gov.in/dataset/85bb464f-bcd5-440b-be2c-fadb2725df4c/resource/5b0a8a48-.../gwl_tel_6_hourly_maharashtra-gw_mh_2021_2025.csv , 529 MB)
PMGSY works	POST https://pmgsy.dord.gov.in/dbweb/ChiefSecretary/GetRoadTenderDetails (session + anti-forgery token; see backend/load_pmgsy_works.py)
PMGSY GeoSadak	https://github.com/datameet/pmgsy-geosadak (Road_DRRP shapefile by state)
NFHS-5 Maharashtra	https://dhsprogram.com/pubs/pdf/FR374/FR374_Maharashtra.pdf
AwaazIQ API (ours)	GET /villages?district= · GET /villages/{gazetteer_id} · GET /assets/{id} · GET /gazetteer/facilities · GET /investment-alignment · POST /recompute-scores

16. Appendix: sources ruled out, and why

Source	Why it doesn't work for us
Sentinel-1/2 satellites	10 m pixels: can't see potholes, taps, staff or school roofs. Only 1 of 238 road complaints (0.4%) even mentions flooding.
Google Earth Engine	24–48 h account approval and a high risk of false "damage" from clouds or seasons, bad in front of judges.
Bhuvan high-resolution imagery	1 m only for about 200 cities; rural Kolhapur/Nashik get 2.5 m, worse than Planet's 3.7 m-everywhere; login needed.
Mapillary, KartaView, OpenStreetMap	No internal village roads: only 5 mapped through-roads within 3 km of Borhate.
MSRTC bus GPS, PWD pothole portal, HMIS, Meri Sadak	Citizen- or staff-facing apps with no public data.
MORTH accident data	State/national totals only.
NITI Aspirational Districts dashboard	Real-time, but Maharashtra's districts in it are Nandurbar, Osmanabad, Washim, Gadchiroli, not Kolhapur or Nashik.
IMD rainfall (imdpune / data.gov.in)	Connection failed and needs an API key; MOSDAC or CWC/SACHET cover the need.
AI deepfake detection, AI severity scoring	Unreliable and adversarial; photos are evidence for an officer, not an automatic score.

Research sources behind these findings (datasets, papers, portals) are listed with links in `LIVE_GOV_DATA_RESEARCH.md`, `PHOTO_VERIFICATION_RESEARCH.md` and `STALE_INFRA_DEFICIT_RESEARCH.md`.